

Session 1 : Géométrie et Découverte de l'API

💡 Objectifs de la session

Présenter le cours et son fonctionnement, découvrir l'écosystème Babylon.js, rappeler les notions mathématiques essentielles, et construire un maillage 3D à la main pour comprendre la structure intime d'un modèle.

Présentation du cours

Orientations et évaluation

Ce cours explore la programmation graphique 3D temps réel appliquée au jeu vidéo, en utilisant entre autres Babylon.js comme moteur, mais aussi d'autres outils et frameworks (Unity, Unreal Engine, Blender, Godot).

Évaluation :

- Travaux pratiques (TP) : Mise en pratique des concepts de chaque session.
- Projet final : Création d'un projet utilisant l'ensemble des principes abordés.

Approche pédagogique : Chaque session combine théorie et atelier pratique. Le code est écrit en JavaScript/TypeScript et s'exécute directement dans le navigateur. Des démonstrations seront aussi faites sur les engins commerciaux, pour montrer les analogies, la programmation graphique étant indépendante du moteur utilisé.

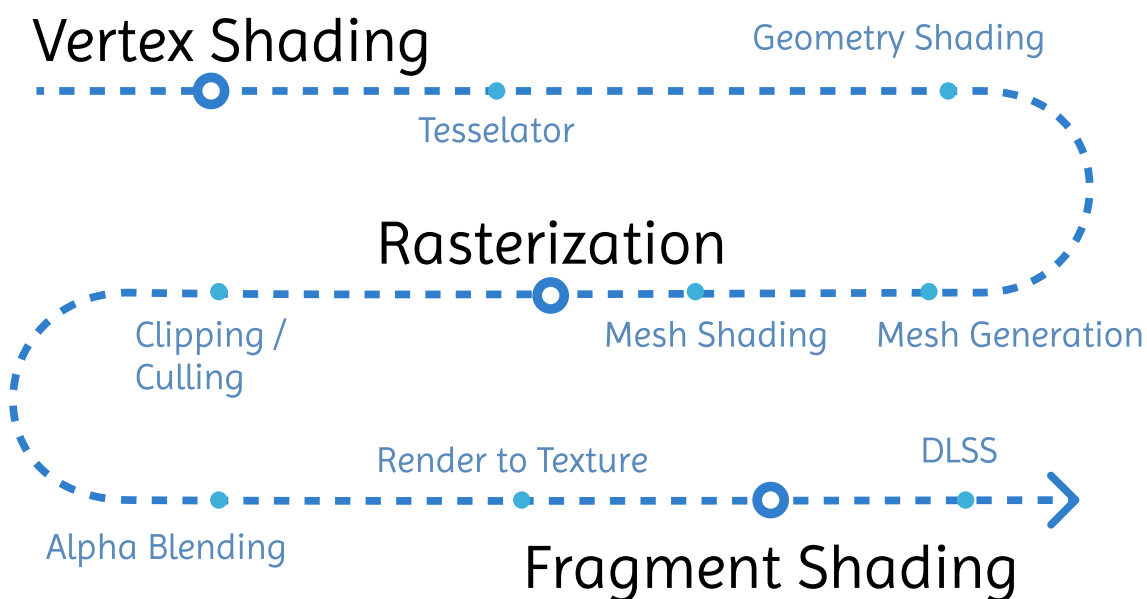


Figure 1: Notre parcours dans l'univers du GPU.

Introduction à la programmation graphique

La programmation graphique consiste à décrire au GPU (carte graphique) comment transformer des données mathématiques (points, vecteurs, matrices) en pixels colorés à l'écran. C'est la rencontre entre les mathématiques, la physique (optique) et l'informatique.

Contrairement à la programmation classique (CPU, séquentielle), la programmation graphique exploite le parallélisme massif du GPU : des milliers de calculs exécutés simultanément.

Découverte de l'environnement : Babylon.js et le Playground

L'écosystème Babylon.js

Babylon.js est un moteur 3D open-source écrit en TypeScript, s'exécutant dans le navigateur via WebGL et WebGPU. Il offre :

- Un moteur de rendu temps réel (PBR, lumières, ombres, post-process).
- Un éditeur visuel de shaders (Node Material Editor).
- Une API complète pour la création de scènes, d'animations et d'interactions.

Avantage clé : Aucune installation requise. Tout s'exécute dans le navigateur, et il est possible de partager facilement les exemples via URL.

💡 Le Babylon.js Playground

Le **Playground** (<https://playground.babylonjs.com>) est l'outil idéal pour prototyper :

- Zéro installation : on écrit du code JS directement dans le navigateur.
- Itération rapide : modification en direct, rendu instantané.
- Partage : chaque scène a une URL unique partageable.
- Modèles : des centaines d'exemples officiels pour apprendre.

C'est l'environnement que nous utiliserons pour les ateliers pratiques.

Rappels mathématiques : Vecteurs et Matrices

Vecteurs (Vector3)

Un vecteur représente une direction et une magnitude dans l'espace 3D. Dans Babylon.js, on utilise la classe `BABYLON.Vector3`.

Opérations essentielles :

```
const a = new BABYLON.Vector3(1, 2, 3);
const b = new BABYLON.Vector3(4, 5, 6);

a.add(b);           // Addition
a.subtract(b);      // Soustraction
a.scale(2);         // Multiplication par scalaire
BABYLON.Vector3.Dot(a, b); // Produit scalaire
BABYLON.Vector3.Cross(a, b); // Produit vectoriel
a.normalize();      // Vecteur unitaire
a.length();         // Magnitude (norme)
```

- **Produit scalaire** : Mesure l'alignement entre deux vecteurs. Utilisé pour l'éclairage (angle entre normale et lumière).
- **Produit vectoriel** : Produit un vecteur perpendiculaire aux deux autres. Utilisé pour calculer les normales de surface.

Matrices (Matrix)

Une matrice 4x4 représente une **transformation** dans l'espace (translation, rotation, scale, ou combinaison). Babylon.js utilise BABYLON.Matrix.

Transformations courantes :

```
// Translation
const T = BABYLON.Matrix.Translation(1, 0, 0);

// Rotation (autour de Y)
const R = BABYLON.Matrix.RotationY(Math.PI / 4);

// Scale
const S = BABYLON.Matrix.Scaling(2, 2, 2);

// Composition : T * R * S (l'ordre compte !)
const world = T.multiply(R).multiply(S);
```

Pourquoi 4x4 ? La 4ème coordonnée (w) permet de unifier translations et rotations dans une seule matrice. Un point (x, y, z) devient $(x, y, z, 1)$ en coordonnées homogènes.

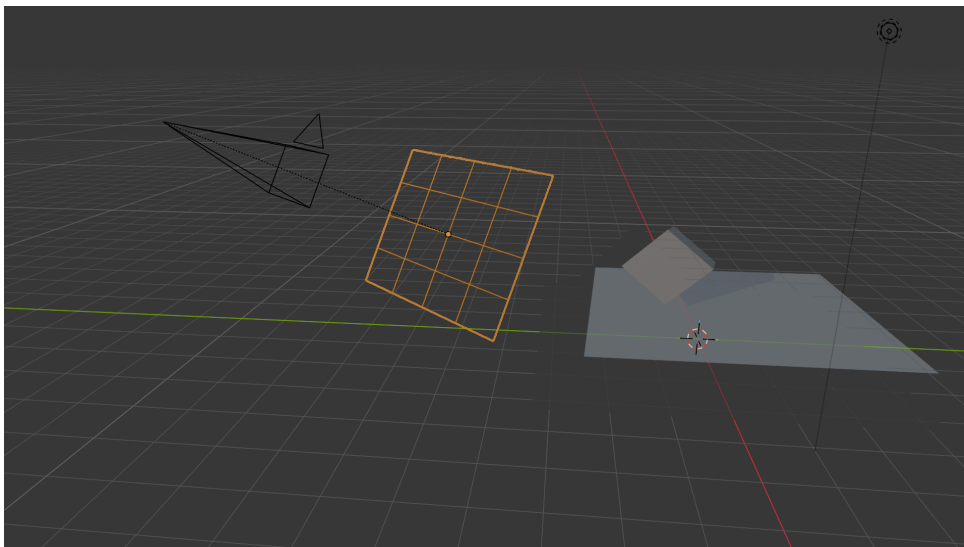


Figure 2: L'écran est une projection du monde en 3 dimensions.

La géométrie bas niveau

Composantes d'un Vertex

Un **vertex** (sommet) n'est pas qu'un point dans l'espace. Il peut contenir plusieurs attributs :

- **Position** : Coordonnées (x, y, z) dans l'espace objet. Obligatoire.
- **Normale** : Vecteur perpendiculaire à la surface, utilisé pour l'éclairage. Direction (nx, ny, nz) .
- **UV** : Coordonnées de texture (u, v) dans $[0, 1]$, reliant le sommet à un pixel de la texture 2D.
- **Tangente** : Vecteur tangent à la surface, utilisé pour le normal mapping (espace tangent).
- **Couleur** : Couleur RGBA par sommet (r, g, b, a) , utilisée pour le vertex coloring.

Ces attributs sont stockés dans des **Vertex Buffers** séparés et envoyés au GPU.

Types de primitives

En graphisme temps réel, toute surface est décomposée en triangles. Les types de primitives GPU sont :

- **Points** : Un point par vertex (rarement utilisé).
- **Lignes** : Une ligne par paire de vertices.
- **Line Strip** : Lignes connectées en chaîne.
- **Triangles** : Un triangle par triplet de vertices. C'est le type dominant.
- **Triangle Strip** : Triangles connectés partageant des arêtes (plus économe en vertices).
- **Triangle Fan** : Triangles partageant un vertex central.

Le triangle est la primitive de base car :

- Trois points définissent toujours un plan (pas d'ambiguïté).
- L'interpolation des attributs (UV, normales) est bien définie à l'intérieur.

Atelier pratique : Création d'un mesh à la main

📌 Objectif de l'atelier

Construire un maillage triangulaire en manipulant directement les **Vertex Buffers** et l'**Index Buffer** via l'objet `VertexData` de Babylon.js. Comprendre la structure intime d'un modèle 3D.

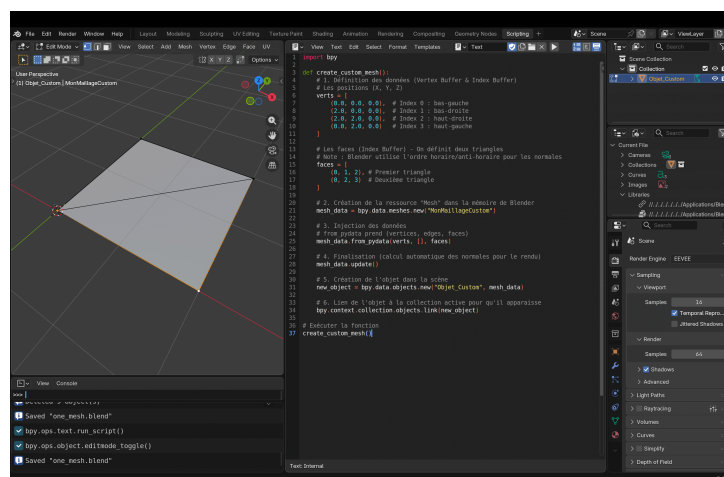


Figure 3: Le même maillage peut être créé visuellement dans Blender — les concepts de vertices, normales et UVs sont universels, indépendants du moteur utilisé.

VertexData : L'objet de construction

BABYLON.VertexData est le conteneur bas niveau pour les données géométriques. Il permet de définir manuellement chaque buffer avant de les appliquer à un mesh.

Buffers principaux :

- **positions** : Tableau de floats ($x_0, y_0, z_0, x_1, y_1, z_1, \dots$)
- **normals** : Tableau de floats ($nx_0, ny_0, nz_0, \dots$)
- **Uvs** : Tableau de floats ($u_0, v_0, u_1, v_1, \dots$)
- **indices** : **Index Buffer** — tableau d'entiers désignant quels vertices forment chaque triangle.

Créer un triangle à la main.

```

// 1. Créer un mesh vide
const customMesh = new BABYLON.Mesh("custom", scene);

// 2. Définir les Vertex Buffers
const positions = [
    -1, -1, 0,    // Vertex 0 : bas-gauche
    1, -1, 0,     // Vertex 1 : bas-droite
    0, 1, 0       // Vertex 2 : haut-centre
];

const normals = [
    0, 0, 1,      // Normale du Vertex 0 (vers la caméra)
    0, 0, 1,      // Normale du Vertex 1
    0, 0, 1       // Normale du Vertex 2
];

const uvs = [
    0, 0,         // UV du Vertex 0
    1, 0,         // UV du Vertex 1
    0.5, 1        // UV du Vertex 2
];

// 3. Définir l'Index Buffer (quels vertices forment les triangles)
const indices = [0, 1, 2]; // Un seul triangle

// 4. Assembler le VertexData
const vertexData = new BABYLON.VertexData();
vertexData.positions = positions;
vertexData.normals = normals;
vertexData.uvs = uvs;
vertexData.indices = indices;

// 5. Appliquer au mesh
vertexData.applyToMesh(customMesh);

// 6. Assigner un matériau
const mat = new BABYLON.StandardMaterial("mat", scene);
mat.diffuseColor = new BABYLON.Color3(1, 0, 0);
mat.wireframe = false;
customMesh.material = mat;

```

L'Index Buffer : Pourquoi et Comment ?

L'**Index Buffer** (ou tableau d'indices) indique au GPU dans quel ordre lire les vertices pour former des triangles.

Pourquoi des indices ?

- Un vertex peut être partagé par plusieurs triangles (ex: un cube a 8 sommets mais 12 triangles).
- Sans index buffer, on dupliquerait les vertices partagés (gaspillage mémoire).
- Avec index buffer : 8 vertices + 36 indices au lieu de 36 vertices.

Convention de winding (sens d'enroulement) :

- Babylon.js utilise le **counter-clockwise** (sens inverse horaire) vu de l'extérieur.
- indices = [0, 1, 2] : le triangle est visible depuis +Z.
- indices = [0, 2, 1] : le triangle est visible depuis -Z (backface culling).

Démonstration : Construire un maillage de A à Z

Script complet pour le Babylon.js Playground. Ce code peut être collé directement dans le Playground. Il illustre la création d'un triangle et d'un carré (Quad) en manipulant les buffers.

```
export const createScene = function () {
    // 1. Initialisation de la scène basique
    const scene = new BABYLON.Scene(engine);

    // Caméra orbitale pour tourner autour de nos créations
    const camera = new BABYLON.ArcRotateCamera("camera", -Math.PI / 2, Math.PI / 2.5,
5, BABYLON.Vector3.Zero(), scene);
    camera.attachControl(canvas, true);

    // Lumière basique
    const light = new BABYLON.HemisphericLight("light", new BABYLON.Vector3(0, 1, 0),
scene);
    light.intensity = 0.7;

    // =====
    // EXEMPLE 1 : LE TRIANGLE
    // =====
    const triangleMesh = new BABYLON.Mesh("monTriangle", scene);

    const positionsTri = [
        -1, -1, 0,    // 0: bas-gauche
        1, -1, 0,    // 1: bas-droite
        0, 1, 0      // 2: haut-centre
    ];

    // Dans Babylon (Left-Handed), normales vers -1 en Z pour faire face à la caméra
    const normalsTri = [
        0, 0, -1,
        0, 0, -1,
        0, 0, -1
    ];

    const uvsTri = [
        0, 0,
        1, 0,
        0.5, 1
    ];

    const indicesTri = [0, 1, 2];

    const vertexDataTri = new BABYLON.VertexData();
    vertexDataTri.positions = positionsTri;
    vertexDataTri.normals = normalsTri;
    vertexDataTri.uvs = uvsTri;
    vertexDataTri.indices = indicesTri;

    vertexDataTri.applyToMesh(triangleMesh);

    // Matériau rouge pour le triangle
    const matTri = new BABYLON.StandardMaterial("matTri", scene);
    matTri.diffuseColor = new BABYLON.Color3(1, 0, 0);
    // matTri.wireframe = true; // Décommenter pour tester
```

```

triangleMesh.material = matTri;

triangleMesh.position.x = -1.5;

// =====
// EXEMPLE 2 : LE CARRÉ (QUAD) AVEC TEXTURE
// =====
const quadMesh = new BABYLON.Mesh("monCarre", scene);

const positionsQuad = [
    -1, -1, 0, // 0 : bas-gauche
    1, -1, 0, // 1 : bas-droite
    1, 1, 0, // 2 : haut-droite
    -1, 1, 0 // 3 : haut-gauche
];

const normalsQuad = [
    0, 0, -1,
    0, 0, -1,
    0, 0, -1,
    0, 0, -1
];

const uvsQuad = [
    0, 0, // 0
    1, 0, // 1
    1, 1, // 2
    0, 1, // 3
];

const indicesQuad = [0, 1, 2, 0, 2, 3];

const vertexDataQuad = new BABYLON.VertexData();
vertexDataQuad.positions = positionsQuad;
vertexDataQuad.normals = normalsQuad;
vertexDataQuad.uvs = uvsQuad;
vertexDataQuad.indices = indicesQuad;

vertexDataQuad.applyToMesh(quadMesh);

// Matériau avec texture pour prouver que les UV fonctionnent
const matQuad = new BABYLON.StandardMaterial("matQuad", scene);
matQuad.diffuseTexture = new BABYLON.Texture("textures/
checkerboard_basecolor.png", scene);
quadMesh.material = matQuad;

quadMesh.position.x = 1.5;

// =====
// OUTILS PEDAGOGIQUES
// =====
// scene.debugLayer.show(); // Afficher l'inspecteur pour montrer les buffers

return scene;
};

```

💡 Points interactifs à observer

1. **Le Backface Culling en direct** : Faites pivoter la caméra dans le Playground pour regarder derrière le triangle et le carré. Les objets deviendront invisibles. Cela illustre parfaitement que l'ordre des indices définit une face orientée d'un seul côté, optimisant ainsi le rendu GPU.
2. **Jouer avec le Wireframe** : Décommentez la ligne `matTri.wireframe = true;`. C'est un excellent moyen visuel de prouver que le carré (à droite) est bel et bien composé de deux triangles (la diagonale apparaît).
3. **Casser l'Index Buffer** : Changez `indicesQuad = [0, 1, 2, 0, 2, 3]` par `[0, 1, 2]`. Un seul triangle du carré s'affiche, prouvant le rôle des indices.
4. **Casser les UVs** : Changez la coordonnée UV du point 2 (haut-droite) de `1, 1` à `0.5, 0.5`. La texture du damier se déformera instantanément, ce qui permet de constater le fonctionnement du "texture mapping".